

▼ Suicide Detection: Fine-Tuning and Evaluation – BERTweet

```
!pip install -q kaggle
from google.colab import files
files.upload()
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!kaggle datasets download -d nikhileswarkomati/suicide-watch
!unzip suicide-watch.zip
import pandas as pd
df = pd.read_csv('Suicide_Detection.csv')
print(f"Dataset shape: {df.shape}")
print(df.head())
```

Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle.json to kaggle.json

Dataset URL: <https://www.kaggle.com/datasets/nikhileswarkomati/suicide-watch>

License(s): CC-BY-SA-4.0

Archive: suicide-watch.zip

inflating: Suicide_Detection.csv

Dataset shape: (232074, 3)

Unnamed: 0	text	class
0	2 Ex Wife Threatening SuicideRecently I left my ...	suicide
1	3 Am I weird I don't get affected by compliments...	non-suicide
2	4 Finally 2020 is almost over... So I can never ...	non-suicide
3	8 i need helpjust help me im crying so hard	suicide
4	9 I'm so lostHello, my name is Adam (16) and I've...	suicide

```
import pandas as pd
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import AutoTokenizer, AutoModelForSequenceClassification, get_linear_schedule_with_warmup
from torch.optim import AdamW
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from tqdm.auto import tqdm

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

class RedditDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels
    def __getitem__(self, idx):
        item = {k: torch.tensor(v[idx]) for k, v in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item
    def __len__(self):
        return len(self.labels)

def encode_data(model_name, train_texts, val_texts):
    tokenizer = AutoTokenizer.from_pretrained(model_name, normalization=True if "bertweet" in model_name else False)
    train_enc = tokenizer(train_texts, truncation=True, padding=True, max_length=128)
    val_enc = tokenizer(val_texts, truncation=True, padding=True, max_length=128)
    return tokenizer, train_enc, val_enc

df = pd.read_csv("Suicide_Detection.csv").dropna()
df['label'] = df['class'].map({'non-suicide': 0, 'suicide': 1})
train_texts, val_texts, train_labels, val_labels = train_test_split(
    df['text'].tolist(), df['label'].tolist(), test_size=0.2, stratify=df['label'], random_state=42
)

def train_and_evaluate(model_name):
    tokenizer, train_enc, val_enc = encode_data(model_name, train_texts, val_texts)
    train_ds = RedditDataset(train_enc, train_labels)
    val_ds = RedditDataset(val_enc, val_labels)
    train_loader = DataLoader(train_ds, batch_size=16, shuffle=True)
    val_loader = DataLoader(val_ds, batch_size=16)
    model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2).to(device)
    optimizer = AdamW(model.parameters(), lr=2e-5)
    total_steps = len(train_loader) * 3
    scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0, num_training_steps=total_steps)
    best_val_loss = float('inf')
    for epoch in range(3):
        model.train()
        loop = tqdm(train_loader, desc=f"vinai/bertweet-base Epoch {epoch+1}/3")
        for batch in loop:
            optimizer.zero_grad()
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            loss = outputs.loss
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            optimizer.step()
            scheduler.step()
            loop.set_postfix(loss=loss.item())
        model.eval()
        preds, true, total_val_loss = [], [], 0
        for batch in val_loader:
            batch = {k: v.to(device) for k, v in batch.items()}
            with torch.no_grad():
                outputs = model(**batch)
            total_val_loss += outputs.loss.item()
            logits = outputs.logits
            preds.extend(torch.argmax(logits, dim=1).cpu().numpy())
            true.extend(batch['labels'].cpu().numpy())
        avg_val_loss = total_val_loss / len(val_loader)
        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            model.save_pretrained(f"best_vinai_bertweet-base")
            tokenizer.save_pretrained(f"best_vinai_bertweet-base")
        report = classification_report(true, preds, target_names=['Non-suicide', 'Suicide'])
        print(report)

train_and_evaluate("vinai/bertweet-base")

print('Label value counts:', df['label'].value_counts())
df['label'] = df['label'].astype(int)
assert df['label'].isin([0, 1]).all(), 'Invalid labels detected!'
assert len(tokenizer) == model.config.vocab_size, 'Tokenizer and model vocab size mismatch'
```

```

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:104: UserWarning:
Error while fetching `HF_TOKEN` secret value from your vault: 'Requesting secret HF_TOKEN timed out. Secrets can only be fetched when running from the Colab UI.'.
You are not authenticated with the Hugging Face Hub in this notebook.
If the error persists, please let us know by opening an issue on GitHub (https://github.com/huggingface/huggingface\_hub/issues/new).
  warnings.warn(

config.json: 100%                               558/558 [00:00<00:00, 69.6kB/s]

vocab.txt: 100%                                843k/843k [00:00<00:00, 2.99MB/s]

bpe.codes: 100%                               1.08M/1.08M [00:00<00:00, 4.94MB/s]

tokenizer.json: 100%                          2.91M/2.91M [00:00<00:00, 9.30MB/s]
emoji is not installed, thus not converting emoticons or emojis into text. Install emoji: pip3 install emoji==0.6.0
pytorch_model.bin: 100%                      543M/543M [00:01<00:00, 541MB/s]
Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at vinai/bertweet-base and are newly initialized: ['classifier.dense.bias', 'classif
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

vinai/bertweet-base Epoch 1/3: 100%            11604/11604 [1:07:59<00:00, 3.05it/s, loss=0.000329]

model.safetensors: 100%                      543M/543M [00:01<00:00, 449MB/s]

      precision    recall  f1-score   support

 Non-suicide       0.98       0.98       0.98     23208
   Suicide       0.98       0.98       0.98     23207

   accuracy       0.98       0.98       0.98     46415
  macro avg       0.98       0.98       0.98     46415
weighted avg       0.98       0.98       0.98     46415


vinai/bertweet-base Epoch 2/3: 100%            11604/11604 [1:07:59<00:00, 3.05it/s, loss=0.000199]

      precision    recall  f1-score   support

 Non-suicide       0.98       0.99       0.98     23208
   Suicide       0.99       0.98       0.98     23207

   accuracy       0.98       0.98       0.98     46415
  macro avg       0.98       0.98       0.98     46415
weighted avg       0.98       0.98       0.98     46415


vinai/bertweet-base Epoch 3/3: 100%            11604/11604 [1:08:00<00:00, 3.04it/s, loss=0.000177]

      precision    recall  f1-score   support

 Non-suicide       0.98       0.99       0.98     23208
   Suicide       0.99       0.98       0.98     23207

   accuracy       0.98       0.98       0.98     46415
  macro avg       0.98       0.98       0.98     46415
weighted avg       0.98       0.98       0.98     46415


Label value counts: label
1    116037
0    116037
Name: count, dtype: int64
```

```

import pandas as pd

bertweet_report = {
    "Non-suicide": {"precision": 0.98, "recall": 0.98, "f1-score": 0.98, "support": 23208},
    "Suicide": {"precision": 0.98, "recall": 0.98, "f1-score": 0.98, "support": 23207},
    "accuracy": 0.98,
    "macro avg": {"precision": 0.98, "recall": 0.98, "f1-score": 0.98, "support": 46415},
    "weighted avg": {"precision": 0.98, "recall": 0.98, "f1-score": 0.98, "support": 46415},
}

pd.DataFrame(bertweet_report).T[["precision", "recall", "f1-score", "support"]]
```



	precision	recall	f1-score	support
Non-suicide	0.98	0.98	0.98	23208.00
Suicide	0.98	0.98	0.98	23207.00
accuracy	0.98	0.98	0.98	0.98
macro avg	0.98	0.98	0.98	46415.00